DOD Task 1

Premisa task-ului:

Scopul primului task a fost de a realiza o aplicatie capabila sa simuleze miscarea si coliziunea unui numar foarte mare de obiecte folosind doua paradigme arhitecturale diferite **OOP vs DOD**, cu precizarea de a observa diferentele in performanta simularii.

Cum am facut noi asta?

Am ales sa folosim **Visual Studio Code** ca IDE, lucrand in **C++** si folosind bibliotecile **Raylib**, pentru game logic (inregistrarea apasarilor de buton, controlul frame rate-ului, desenarea obiectelor etc.) si **ImGui** pentru toate interfata interactiva grafica, impreuna cu toate Quality of Life realizate pentru aceasta.

Am creat un sablon pentru munca noastra: un fiesier main.cpp, benchmark.cpp, config.h, dod_system.h si oop_system.h. Astfel separand munca noastra ca echipa. Dupa realizarea celor doua sisteme OOP si DOD am trecut la prima iteratie de spawnare si coliziune a obiectelor unde am intampinat prima problema.

Problema: **Programul crapa la 20.000** de obiecte. Logica modului de **coliziune trebuia schimbata**.

Initial treceam prin toate obiectele de doua ori comparand element cu element si verificand daca exista o coliziune intre cele doua. In cazul in care exista rezolvam acest lucru schimband viteza si directe intre cele doua obiecte.

```
void ResolveCollisions(){
    int count = (int)posX.size();
    for (int i = 0; i < count - 1; i++) {
        for(int j = i + 1; j < count; j++)
        {
            //Calcul distanta euclidiana intre doua cercuri
            float dX = posX[j] - posX[i];
            float dY = posY[j] - posY[i];
            float distanceSq = dX * dX + dY * dY;
            float radiusSum = radius[i] + radius[j];
            float radiusSumSq = radiusSum * radiusSum;

            //verific patratul distantelor ca e mai cost efficient decat sa fac sqrt
            //fac verificarea > 0.0001f ca sa nu am eroare la impartirea la 0
            if(distanceSq < radiusSumSq && distanceSq > 0.0001f) {
                //coliziune cu efect elastic
                float tempVx = velX[i];
                float tempVy = velY[i];
                // schimbam vitezele intre cele doua cercuri
                velX[i] = velX[j];
                velY[i] = velY[j];
                velX[j] = tempVx;
                velY[j] = tempVy;
            }

        }
    }
}
```

Varianta folosita era foarte lenta si facea o multitudine de comparari fara sens. De exemplu daca un obiect se afla in coltul stanga sus al ferestrei noastre, iar al doilea obiect se afla in coltul din dreapta jos, chiar daca un om ar vedea usor faptul ca distanta dintre cele doua este asa de mare incat ele nu ar putea sa se ciocneasca, totusi algoritmul nostru nu lua in calcul acest lucru si tot le verifica pe ambele, iar la un numar imens de obiecte toate comparile inutile se adunau si facea programul sa nu mai mearga.

Solutia:

Pentru a evita verificarea ineficienta a coliziunilor dintre fiecare bila cu toate celelalte, **am impratit fereastra intr-un vector de celule (Grid)**:

- Alocare dinamica: **Fereastra este divizata** in celule de dimensiuni fixe
- Optimizare: In loc de o verificare globala de tip N^2, **coliziunile sunt verificate strict local**

Celula este salvata ca un vector care retine indicele obiectelor continute. Incadram obiectele in functie de pozitia curenta a originii obiectului, cu verificarea, daca luand raza "bilei" aceasta ar suprapune alte celule. Daca da, scriem obiectul in acea celula chiar daca originea nu se afla acolo. Astfel avem obiecte care se pot afla in mai multe celule simultan. Asta ne scapa de multitudinea de verificari inutile facute anterior.

Acum programul merge bine chiar si cu 150.000 de obiecte!

Dar Game Loop?

Fiecare frame ruleaza exact aceeasi pasi:

1) **Verificare Input / Spawn Mouse:** Detecteaza inputului utilizatorului si spawneaza un numar setat de bile pe o raza configurabila in jurul cursorului.
2) **Clear Grid:** Golirea vectorilor din celulele gridului pentru a pregati frame-ul curent.
3) **Move & Populate Grid**: Actualizarea miscarii fiecarei bile (inclusiv coliziunile cu marginile ferestrei) si introducerea indicilor in celulele corespunzatoare din grid.
4) **Resolve Collisions**: Parcurgerea celulelor din grid si aplicarea coliziunilor elastice..
5) **Draw & Render**: Clear ecranului, desenarea texturata a bilelor pe baza culorilor si randarea elementelor de interfata ImGui.

Cum am reusit sa impartim munca intre noi?

Pentru realizarea primul task, dat fiind scopul, am creat un **server de discord** pentru a putea face **meet-uri** si a avea un mod confortabil de a **comunica si partaja ecrane**, facand, de altfel, **canale separate** pentru a putea posta materialele educative folosite, concepte de implementat, reguli de respectat, nevoi personale pentru realizarea unor task-uri din partea altor colegi etc.

Toate acestea ne-au permis sa ne impartim munca intre noi fara a exista conflicte sau orice alt impediment.

OK… si concret, cine ce a facut?

Bercea Mihnea: main + benchmark + logica de grid + gravitatie.

Am coordonat impartirea taskurilor si m-am ocupat de retusari pe ici pe colo. Am facut main-ul care foloseste functiile celorlalti colegi si are game loop-ul principal, explicat mai sus. Aici m-am asigurat ca testul e cat mai fair si am facut parcurgerea la fel si pentru OOP si pentru DOD, functiile apeland un singur element de la un anumit index. Am facut si UI-ul cu imgui, unde am adaugat un feature de spawn pe click intr-o raza, ceva basic, si benchmarkul care masoara latenta si fps-urile (basically durata unui gameloop), si face un grafic dragut.Am venit cu idea pentru aceasta metoda de grid, dar in urma rezultatelor obtinute sunt cateva imbunatatiri care mai pot fi facute, dar nu am avut timp. De exemplu, cand pornesc gravitatia, toate bilele se strang spre podea, deci sunt folosite doar celulele de grid din partea de jos, si se face lag. La gravitatie am modificat functiile de miscare din OOP si DOD, si am adaugat o forta care trage mingea in jos si atunci cand loveste podeaua isi pierde din viteza (sa fie bouncy). Am udpatat putin si logica de coliziune pentru a avea ceva mai realist, cu vectori de directie, astfel incat atunci cand 2 bile au coliziune, isi schimba velocitatea in functie de directia pe care o aveau si unde s-au ciocnit.

Nica Octavian: dod_system.h + Velocity Heatmap
Spre deosebire de abordarea clasica OOP (unde fiecare obiect isi tine datele izolate), in dod_system.h am organizat datele sub forma de vectori (SoA):

Am folosit 6 vectori cu nume descriptivi: posX, posY, velX, velY, radius, colors. Pentru a putea determina toate atributele unui singur obiect este suficient sa ma uit doar la pozitia din fiecare vector al elementelor care ma intereaseaza. De exemplu pentru primul obiect ca sa ii determin pozitia curenta ma uit in vectorii posX si posY la indicele 0.
Functii:

1) SpawnOne: Imi permite sa spawnez un singur obiect caruia ii dau prin parametrii atributele. Foloseste functia de push_back din C++ pentru a adauga in vectorii mei elemente noi.
2) Spawn: Genreaza un numar setat de obiecte dar cu atribute random. Are nevoie doar de count, dimensiunea ferestrei, gradul vitezei random (la cat sa imparta numarul de rand)
3) Despawn: Face diferenta dintre parametrul newCount si size-ul curent al vectorului meu posX care reprezinta numarul de obiecte curente si sterge atatea obiecte luand cate un numar random care indica o pozitie din vector folosind RemoveParticleAt
4) RemoveParticleAt: Am facut un mic trick pentru eficienta, pune pe pozitia la care trebuie sters un element, ultimul element din vector si face pop_back() la ultimul element
5) Move: Updateaza pozitia curenta a tuturor boiectelor facand doar operatiile posX += velX; posY += velY; si logica de coliziune cu border-ul (inverseaza directia vitezei inmultind-o cu -1)
6) MoveDOD: Asemanator cu Move() doar ca pentru un singur obiect, avand ca parametru un indice
7) DrawRGB: foloseste functii din Raylib pentru a desena obiecte dupa textura
8) Getteri si Setteri pentru fiecare atribut

Pentru a urmari comportamentul fizic al particulelor, am creat functia UpdateVelocityHeatmap(int maxSpeed), care coloreaza dinamic fiecare bila in functie de viteza sa curenta:

- Optimizare matematica: In loc sa folosesc functia costisitoare sqrt() pentru a calcula viteza, lucrez direct cu patratul acesteia (speedSq = vx * vx + vy * vy)
- Dinamica raportata la UI: Functia preia valoarea maxSpeed setata de utilizator prin sliderul din interfata.
- Factor de compensare: Pentru a contracara micile efecte de crestere a energiei din coliziunile elastice (unde in timp bilele ar putea ramane blocate in rosu aprins), am introdus un factor de scalare de 15% (effectiveMaxSpeed = maxSpeed * 1.15f). Acest lucru permite ca spectrul culorilor (de la albastru - lent, la verde si rosu - rapid) sa fie distribuit echilibrat si natural pe tot parcursul simularii.
- Generarea spectrului RGB: Valoarea normalizata a vitezei (t) este convertita matematic in canale de culoare RGB.

Sandu Bianca: logica initiala de coliziune in collision.h + logica click dreapta mouse folosita ulterior + functia de magnetism (care foloseste logica anterioara)

Am realizat ideea principala a algoritmului de coliziune, care este utilizat pentru ambele implementari de paradigme. Aplicand logica pe celule de grid, functia de coliziune preia vectorul de indici ai bilelor si paradigma de utilizat pentru stocarea acestora. Bilele sunt parcurse una cate una, in perechi, in fiecare celula cu ajutorul vectorului de indici.
Suprapunerea intre doua bile este verificata cu formula: $dX^2 + dY^2 <= r^2$, unde dX si dY sunt distantele calculate dintre centrele celor doua bile, iar r este raza unei bile (care e egala pentru toate bilele). Functia calculeaza doi vectori de directie (pe cele doua axe) de la bila 1 la bila 2, si in functie de cat se suprapun bilele la coliziune, calculeaza distanta parcursa in miscare si inverseaza directia de miscare, adaugand si un impuls calculat. Functia originala era mai simplista, si a fost completata si modificata ulterior.

Functia de magnetism preia datele in functie de paradigma folosita si calculeaza vectori de directie noi pentru bilele care se afla in raza cercului determinat de cursor (logica din benchmark) in functie de viteza bilelor initiala si de distanta dintre bile si cursor. Viteza de miscare a bilelor este o valoare calculata in functie de distanta fata de cursor, astfel ca va fi mai mica odata ce se aproprie de cursor, bila incetinind. Vectorii de directie le-am luat valori intregi, rotunjiri ale formulei de versor * viteza de miscare.
Functia aceasta, impreuna cu cea de repulsion/respingere, le-am apelat in logica de verificare a mouse-ului, si sunt distinse printr-o variabila de toggle care se schimba in benchmark. Functiile stau active sau inactive cu ajutorul unei alte variabile de toggle, schimbata la click dreapta.

Sindelaru Stefan: Eu m-am ocupat de jumatatea de sistemul de obiecte OOP, format dintr-o ierarhie de 3 clase: obiect, actor, si intr-un final Bila. Sistemul OOP dispune de aceleasi functii ca cel DOD: o functie pentru creearea unei bile cu datele primite (care, cu adevarat doar apeleaza constructorul clasei Bila si impinge obiectul intr-un vector ce contine toate

obiectele de pe ecran), o functie pentru miscatul unei bile, o functie pentru desenatul bilelor in spatiu, o functie pentru stersul aleator de obiecte pana cand raman doar N, si de asemenea aceeasi getteri si setteri pentru obiectele in sine;
Feature-ul meu personal este opusul magnetismului colegei mele, anume o functie prin care, atunci cand utilizatorul apasa pe right-click, bilele sunt impinse in exteriorul razei mouse-ului, cu forta invers porportionala fata de cat de aproape de afla de centrul zonei; Cu cat sunt mai departe, cu atat mai slab sunt impinse.
Calculul pentru acest feature este similar cu cel pentru magnetism; Se calculeaza distanta de la centru, apoi se imparte in doi vectori ce vor reprezenta noile viteze X si Y. Diferenta dintre functia de atragere si de respingere este in principal semnul, unde la magnetism viteza veche a obiectului se suprascrie cu viteza noua calculata prin inmultirea modulului distantei pe o anumita coordonata carteziana impartita la distanta absoluta si apoi inmultita cu viteza, functia de respingere suprascrie viteza obiectului cu opusul aceleiasi formule.